ITSON · INGENIERÍA EN SOFTWARE

VetSync

Sistema de Gestión de Citas y Telemedicina Veterinaria
Ingeniería de Software potenciada por IA

Alumno	Julian Espinosa Llanos
Matrícula	260306
Materia	Optativa I: Fundamentos de la Inteligencia Artificial en la Ingeniería de Software
Entrega	8 de noviembre de 2026
Contenido	1. Enlace al código fuente · 2. Bitácora de prompts · 3. Reflexión técnica · Anexo: evidencia

1. Enlace al código fuente

Repositorio en GitHub: https://github.com/Junsa64/vetsync

Monolito modular en Node.js 22 + TypeScript (Express 5) con frontend en React 19 + Vite. Cuatro módulos desacoplados (`patients`, `vets`, `appointments`, `notifications`), TDD con historial rojo → verde en los commits, reglas SOLID verificadas con pruebas de arquitectura y CI en GitHub Actions que exige 80 % de cobertura. Las instrucciones para correrlo están en el `README.md`. La CI de GitHub Actions corre en cada push: ESLint, 59 pruebas con Express real, cobertura y build del frontend, en verde.

59	99.37 %	4	3 / 3
pruebas en verde	cobertura de líneas	módulos con puertos propios	requerimientos de negocio

2. Bitácora de prompts

Herramienta en los 5 prompts: Claude Opus 5.5 en claude.ai, en modo agente (con acceso a terminal y archivos, por lo que podía ejecutar las pruebas que escribía).

Cómo se trabajó

El alumno entregó el caso de uso, eligió el stack (Node + Express + React) y le dio a la IA el documento de la metodología, con la instrucción de seguirla. Los prompts de esta bitácora son los de ese documento, adaptados a VetSync, y la IA los ejecutó en modo agente. Cada resultado se pasó por tres filtros antes de aceptarse: **las pruebas** (sin una prueba que falle primero no hay implementación), **las reglas de** `AGENTS.md` y **la ejecución real** (renderizar el diagrama, correr el servidor, abrir la app en un navegador). Las críticas describen lo que realmente ocurrió y cómo se detectó cada error. El historial completo está en los commits: los de tipo `test:` muestran la fase roja antes de cada implementación.

#	Prompt	Fase	Hallazgo principal
1	Arquitectura y esquema MySQL	I.1	Script SQL que no corre (FK a una tabla aún no creada) y diagrama ER con sintaxis inválida
2	Archivo de reglas `AGENTS.md`	I.2	Reglas que nadie verifica no sirven: se convirtieron en pruebas y linter
3	Pruebas fallidas de `agendarCita` (TDD)	II.1	Jest no se podía instalar; faltó el caso más peligroso (concurrencia)
4	Refactor SOLID	II.3	La IA violó en el código la regla OCP que ella misma escribió en `AGENTS.md`
5	Code review de seguridad y rendimiento	III.1	**Condición de carrera**: dos citas en el mismo horario del mismo veterinario

Prompt 1 — Diseño de arquitectura (Fase I.1)

Prompt exacto:

> Actúa como un Arquitecto de Software Senior. Para el caso de uso 'VetSync', propón un esquema de base de datos relacional (MySQL) y la estructura de directorios para un backend en Node.js. Justifica tus decisiones basándote en escalabilidad y principios SOLID.

Respuesta de la IA (extracto de la primera versión de `docs/schema.sql`):

```
-- ── Módulo patients ──
CREATE TABLE owners ( ... );
CREATE TABLE pets ( ... CONSTRAINT fk_pets_owner FOREIGN KEY (owner_id) REFERENCES owners (id) );
CREATE TABLE medical_records (
  ...
  CONSTRAINT fk_medrec_vet FOREIGN KEY (vet_id) REFERENCES vets (id) ON DELETE SET NULL
);
-- ── Módulo vets ──
CREATE TABLE vets ( ... );   -- se creaba DESPUÉS de ser referenciada
```

Y en `docs/arquitectura.md`, el diagrama ER:

```
OWNERS { char36 id PK; varchar full_name; varchar email UK; varchar phone }
```

Además propuso: monolito modular con 4 módulos (`patients`, `vets`, `appointments`, `notifications`) en capas `domain / application / infrastructure`, UUID como llave, `ends_at` almacenado e índice compuesto `(vet_id, status, starts_at, ends_at)` para buscar choques.

Crítica:

- **Error que habría tronado en MySQL**: `medical_records` declaraba una llave foránea hacia `vets` antes de que existiera la tabla (MySQL responde con el error 1824). Se detectó releyendo el script en el orden en que se ejecuta. **Cambio:** mover `vets` al inicio.
- **Sintaxis inventada en el diagrama**: Mermaid no acepta atributos separados por `;` en una sola línea dentro de un `erDiagram`. Se veía bien como texto, pero al renderizarlo con `mermaid-cli` falló. **Cambio:** un atributo por línea.
- **Monolito modular en vez de microservicios**: el caso permitía ambos. Con 4 módulos y una sola persona, los microservicios añadían red y despliegues sin beneficio; a cambio, cada módulo tiene sus propios puertos para poder separarlo después.
- **Lo que se aceptó sin cambios** y por qué: almacenar `ends_at` en lugar de solo la duración (la consulta de choques usa el índice sin calcular columnas) y la nota de que MySQL no tiene restricciones de exclusión por rango, así que la regla de choques debe vivir en la aplicación con `SELECT … FOR UPDATE`. Esto último resultó clave en el prompt 5.

Prompt 2 — Contexto del IDE: `AGENTS.md` (Fase I.2)

Prompt exacto:

> Genera un archivo agents.md estricto para un proyecto en TypeScript. Incluye reglas para: nombrado de variables en camelCase, uso obligatorio de interfaces, manejo de errores con try/catch y prohibición de usar 'any'.

Respuesta de la IA (extracto de `AGENTS.md`):

```
- **Prohibido usar `any`** (explícito o implícito). Usa `unknown` y estrecha el tipo.
- **O**: nuevas modalidades o canales se agregan con nuevas implementaciones de un puerto, no con `if`/`switch`.
- **D**: uso obligatorio de interfaces. Los casos de uso reciben sus dependencias por constructor.
- Prohibido `Date.now()` / `new Date()` sin argumentos en dominio o aplicación: usa el puerto `Clock`.
- **Todo manejador HTTP usa `try/catch`** y delega el error con `next(error)` al middleware central.
- Un módulo solo puede importar de otro módulo a través de su `public.ts`.
```

Crítica:

- **Un archivo de reglas no obliga a nada.** La IA lo lee, pero nada impide que lo ignore (y lo ignoró: ver prompt 4). **Cambio:** cada regla importante se volvió verificable:
  - `any` → regla `@typescript-eslint/no-explicit-any` + prueba de arquitectura que busca `any` en `src/`.
  - `new Date()` sin argumentos → regla `no-restricted-syntax` de ESLint + prueba de arquitectura.
  - Imports entre módulos solo por `public.ts` → prueba `moduleBoundaries.test.ts` que recorre los imports.
- Se agregaron reglas que el prompt no pedía pero que necesitaba el caso: `DomainError` con `code` estable (para que el frontend no dependa del texto del mensaje), validación en el borde y límite de tamaño del JSON.
- **Ambigüedad detectada al final**: decía que la raíz de composición era `main.ts (container.ts)`; en realidad es `container.ts`. Una regla ambigua para la IA es una regla que no se cumple, así que se corrigió.

Prompt 3 — TDD asistido: pruebas antes del código (Fase II.1)

Prompt exacto:

> Escribe 5 casos de prueba unitarios usando Jest para la función 'agendarCita'. Incluye casos de borde: veterinario ocupado, fecha en el pasado, y mascota no registrada. No escribas la implementación aún, solo los tests.

Respuesta de la IA (extracto de `test/appointments/scheduleAppointment.test.ts`, commit `c542113`):

```
it('rechaza la cita si el veterinario ya está ocupado en ese horario', async () => {
  const { useCase } = createSut();
  await useCase.execute(command({ startsAt: at('2026-11-11T16:00:00.000Z'), durationMinutes: 60
}));

  await assert.rejects(
    useCase.execute(command({ petId: 'pet-michi', startsAt: at('2026-11-11T16:30:00.000Z') })),
    isDomainError(ConflictError, 'VET_BUSY'),
  );
});

it('permite citas contiguas: el fin de una es el inicio de la otra', async () => { ... });
```

Generó 12 pruebas (los 5 casos pedidos más contiguas, envolvente, otro veterinario, misma mascota con dos veterinarios, duración inválida, telemedicina y evento publicado) y un esqueleto que lanzaba `No implementado`. Resultado al correrlas: **0 de 12 pasan** (`docs/evidencia/01-tdd-rojo.txt`). Después de la implementación: 12/12.

Crítica:

- **Se cambió Jest por `node:test`.** El entorno donde trabajaba la IA no tenía red para instalar paquetes; usar Jest significaba entregar pruebas que nadie había ejecutado. `node:test` viene con Node 22, no agrega dependencias y soporta umbrales de cobertura. Quedó escrito en `AGENTS.md §6`.
- **Se aceptaron 12 en vez de 5** porque cada extra prueba una regla real. La de citas contiguas es la más valiosa: obliga a usar intervalos semiabiertos `[inicio, fin)`; con `<=` en lugar de `<`, una cita de 16:00–16:30 bloquearía otra a las 16:30.
- **Reloj inyectado** (`FixedClock`): una prueba de "fecha en el pasado" que usa la hora real se rompe sola con el tiempo.
- **Lo que faltó y nadie notó en esta fase**: ninguna prueba simulaba dos solicitudes al mismo tiempo. Las 12 pasaban y aun así el sistema permitía choques (prompt 5). Pruebas en verde no significan código correcto.

Prompt 4 — Refactorización SOLID (Fase II.3)

Prompt exacto (aplicado sobre `scheduleAppointment.ts` de la fase verde):

> Revisa este bloque de código. Identifica violaciones a los principios SOLID y refactorízalo para inyectar las dependencias correctamente. Explica los cambios.

Código revisado (generado por la misma IA en la fase verde, commit `6b7196a`):

```
const id = this.deps.ids.next();
const meetingUrl = command.modality === 'TELEMEDICINE'
  ? await this.deps.meetingRooms.createRoomUrl(id)
  : null;
```

Respuesta de la IA (refactor, commit `efb2a3b`):

```
export type ModalityHandlers = Record<AppointmentModality, ModalityHandler>;
// ...
const { meetingUrl } = await this.deps.modalities[command.modality].prepare(id);
```

Explicó dos violaciones: **OCP** (agregar "consulta a domicilio" obligaba a editar el caso de uso) y **SRP** de nivel método (`execute()` de unas 60 líneas validaba, revisaba choques, construía, guardaba y publicaba). Extrajo `assertSchedulable`, `assertNoClashes` y `publishScheduled`.

Crítica:

- **La alucinación aquí fue de consistencia**: la IA escribió en `AGENTS.md` "nada de `if` por modalidad" y poco después generó justo ese `if`. Se detectó revisando el código contra las reglas, no por las pruebas (que pasaban). Por eso no basta con darle reglas a la IA: hay que revisar.
- **Se aceptó el refactor** porque el tipo `Record<AppointmentModality, …>` hace que TypeScript marque error si alguien agrega una modalidad sin su handler: la regla se cumple en compilación.
- **Cómo se verificó que no cambió el comportamiento**: las 12 pruebas de negocio siguieron pasando sin modificar sus afirmaciones; solo cambió cómo se construye el caso de uso en la preparación de la prueba.
- En el mismo commit se agregaron **pruebas de arquitectura**, para que el próximo `if` o el próximo import cruzado rompa la CI en lugar de depender de que alguien lo vea.

Prompt 5 — Code review de seguridad y rendimiento (Fase III.1)

Prompt exacto:

> Actúa como un experto en Ciberseguridad y Rendimiento. Revisa el siguiente código en busca de: 1) Vulnerabilidades (SQLi, XSS), 2) Fugas de memoria, 3) Queries ineficientes (N+1). Sé crítico y sugiere correcciones.

Respuesta de la IA (resumen; completo en `docs/code-review.md`):

1. **Alta — condición de carrera "revisar y luego guardar"** en `agendarCita`: entre buscar choques y guardar hay `await`, así que dos solicitudes simultáneas pasan la revisión y ambas se guardan.

2. **Media — N+1** en recordatorios: por cada cita se consultaba el contacto del dueño y el nombre del veterinario.
3. **Media — crecimiento sin límite**: la bandeja del correo simulado nunca descartaba mensajes.
4. SQLi y XSS revisados sin hallazgo (no hay SQL en ejecución; React escapa el texto; el único `href` lo genera el servidor).

Crítica:

- **El hallazgo 1 no se aceptó solo porque la IA lo dijera: se demostró.** Se escribió una prueba que lanza dos agendamientos al mismo tiempo para el mismo veterinario y horario. Resultado: **se guardaron las dos citas** (`actual: 2, expected: 1`, `docs/evidencia/06-carrera-rojo.txt`). Era una violación directa del requerimiento 2 que las 12 pruebas del prompt 3 no vieron. **Cambio:** `KeyedMutex` por `vet:<id>` y `pet:<id>`, adquiridos en orden fijo para evitar interbloqueos; la revisión y el guardado ahora son atómicos. La prueba pasó y quedó como regresión.
- **La corrección también se revisó como posible fuga**: un mutex por llave puede dejar llaves retenidas en su mapa interno. Se agregó una prueba que verifica que no quede ninguna (`pendingKeys === 0`) al terminar.
- **Se descartaron** dos cambios que suelen recomendarse por costumbre: "usar un ORM para evitar SQLi" (no hay SQL ejecutándose) y "validar con zod" (los validadores propios ya están probados y evitan otra dependencia).
- **Lo que la IA no puede saber**: el mutex protege una sola instancia del servidor. Con varias réplicas el bloqueo tiene que vivir en la base de datos (`SELECT … FOR UPDATE`). Quedó documentado como riesgo aceptado en vez de esconderlo.

Otros errores de la IA detectados durante el proyecto

Dónde	Qué pasó	Cómo se detectó	Qué se hizo
Correos y frontend	El texto decía "8:45 p.m.." (doble punto): la plantilla ponía punto final y el formato es-MX ya termina en "p.m.". **La IA repitió el mismo error** en el mensaje de éxito del frontend.	Revisando las capturas de la app, no las pruebas (que buscaban la hora con una regex y pasaban).	Fecha en su propia línea y una aserción que prohíbe `..`.
CI (Fase III.2)	El YAML típico usa `npm ci` y `cache: npm`, que fallan sin `package-lock.json`.	Revisando qué archivos existían realmente en el repo.	`npm install` en la CI, con nota para cambiar a `npm ci` al versionar el lockfile.
CI, primera corrida en GitHub	La IA usó `actions/checkout@v4` y `actions/setup-node@v4`: versiones de su entrenamiento que GitHub ya marca como obsoletas (Node 20).	Las 2 advertencias de la primera corrida real de GitHub Actions (que sí pasó en verde).	Actualizadas a v5 y runner fijo en `ubuntu-24.04`.
Compuerta de cobertura	Al probarla con umbral 99.9 % el comando no falló y pareció que el umbral no funcionaba.	Reproduciendo el caso mínimo: Node trunca el umbral a entero (99.9 → 99) y la cobertura era 99.34 %. Con 100 sí falla.	Nada: no era un bug. Investigar antes de "arreglar" evitó romper algo que funcionaba.
Andamiaje de pruebas	El simulador local de Express (usado solo para probar sin red) entraba a los sub-routers aunque hubiera un error de JSON.	La prueba de "JSON inválido" falló con el código equivocado.	Se corrigió el simulador. Un doble de prueba también puede "alucinar" comportamiento.

3. Reflexión técnica

¿Dónde redujo la IA la carga cognitiva y dónde la aumentó? ¿Cómo se evitó la deuda técnica?

Dónde la redujo

La IA quitó casi todo el trabajo mecánico. Los repositorios en memoria, los validadores de entrada, las rutas de Express y la documentación de la API son código predecible que habría tomado horas escribir a mano; la IA los produjo en minutos y con un estilo consistente. También redujo la carga al pensar casos de prueba: para `agendarCita` propuso doce, entre ellos uno que no era obvio, el de las citas contiguas, que obligó a definir los horarios como intervalos semiabiertos. Y en el diseño inicial dio un punto de partida completo (esquema, índices, estructura de carpetas) que solo había que cuestionar, no inventar.

Dónde la aumentó

La carga no desapareció: se movió de escribir a verificar, y verificar resultó más difícil. El ejemplo más claro es la condición de carrera. Las doce pruebas de agendamiento pasaban y el código se veía correcto; aun así, dos solicitudes simultáneas guardaban dos citas en el mismo horario del mismo veterinario, justo lo que el requerimiento prohíbe. Solo apareció al pedir una revisión crítica y, sobre todo, al escribir una prueba que lo demostrara. Código que parece correcto exige más atención que código que se ve mal.

También aumentó al vigilar la consistencia. La IA escribió en `AGENTS.md` que no debía haber `if` por modalidad y poco después generó ese `if`; cometió el error del doble punto en la fecha dos veces, en el correo y en el frontend. Hubo además una falsa alarma: la compuerta de cobertura pareció no funcionar y resultó que Node trunca el umbral a entero. Investigarla costó tiempo, pero evitó "arreglar" algo que funcionaba.

Cómo se evitó la deuda técnica

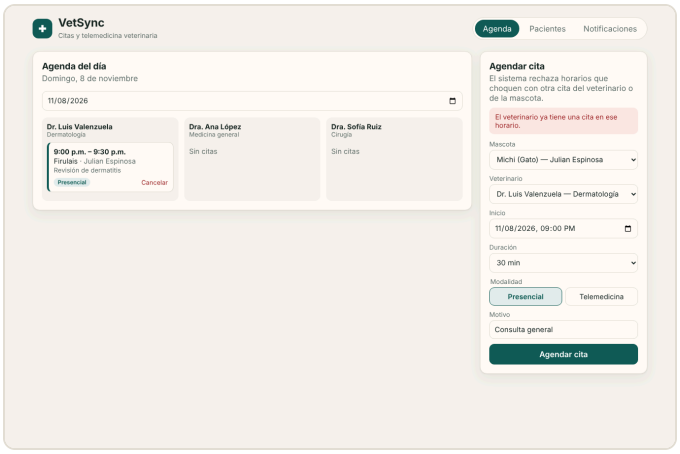
La regla fue no aceptar nada solo porque la IA lo dijera. En concreto:

1. **TDD con evidencia.** Cada módulo tiene un commit con sus pruebas fallando antes de la implementación.
2. **Reglas ejecutables, no solo escritas.** Lo importante de `AGENTS.md` se convirtió en ESLint (sin `any`, sin `new Date()` en el dominio) y en pruebas de arquitectura que fallan si un módulo importa las clases internas de otro.
3. **Compuerta en la CI.** GitHub Actions corre linter y pruebas en cada push y falla si la cobertura baja de 80 % (hoy es 99.37 %).
4. **Cada hallazgo de la revisión, demostrado con una prueba** antes de corregirlo; esas pruebas quedaron como regresión.

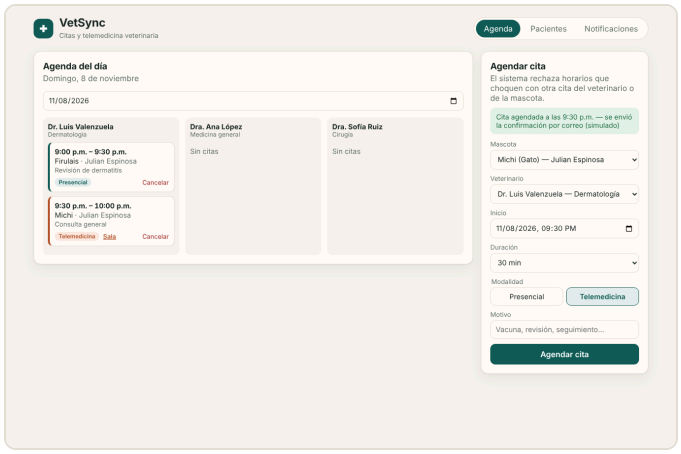
5. **Nada de código sin probar.** El adaptador MySQL no se escribió porque no había forma de probarlo; el esquema y la interfaz quedaron listos. Un adaptador sin pruebas habría sido deuda escondida.
6. **Deuda visible.** Lo que falta (autenticación, persistencia real, bloqueo distribuido) está escrito como riesgo aceptado en `docs/code-review.md`, no oculto.

La lección coincide con la de la clase: la IA es un escritor de código muy rápido, pero el criterio para decidir qué código merece quedarse no se puede delegar.

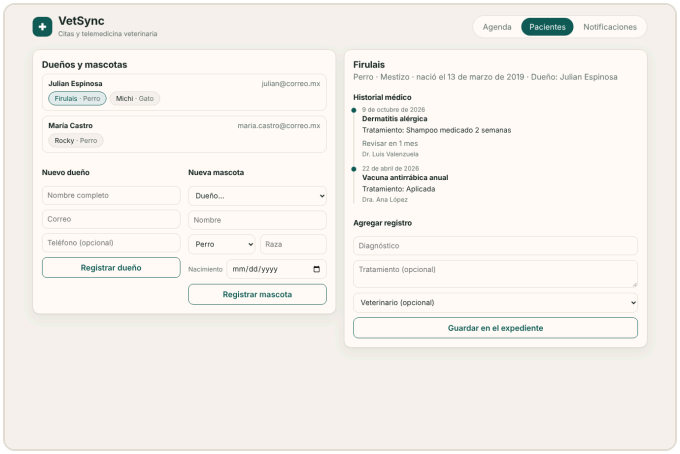
Anexo: evidencia de funcionamiento



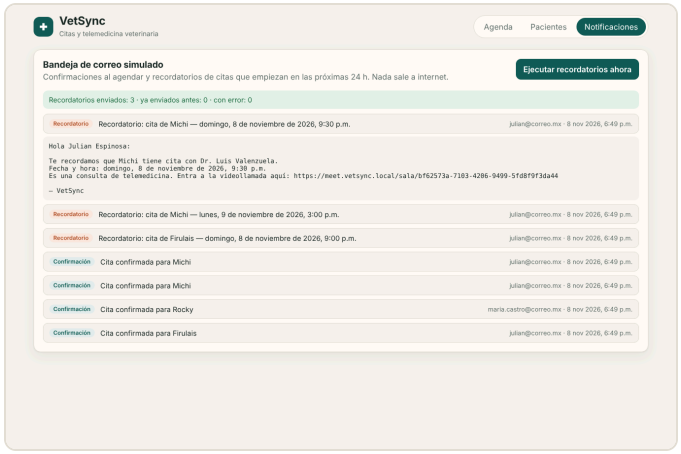
Choque de horario rechazado (HTTP 409 VET_BUSY).



Cita de telemedicina agendada justo al terminar la anterior.



Expediente de la mascota con historial médico.



Bandeja de correo simulado: confirmaciones y recordatorios.